

simultaneously updating weights, i.e. upon detection of a Contradiction, inference is arrested and parameters are updated from the contradiction source to the leaves. This prevents or constrains the propagation of contradictions further into the network.

In contrast to normal neural network training where parameter updates are network-wide, the LNN allows independent formulae, disconnected subgraphs or partitioned subgraphs to learn parameter updates independently, i.e. subgraph inference and parameter convergence may prevent select nodes from future updates and thereby reduce computational idling and improve the convergence rate.

Given classical inputs, the expected classical outputs for a self-supervised LNN is equivalent to that of a supervised LNN, albeit with a possibly longer training time. In stark contrast to ordinary machine learning approaches, the LNN and its dynamic activation function direct the network neurons towards logical self-consistency, without requiring many labeled data points to do so.

G Gradient-transparent clamping

This section describes gradient-transparent clamping used in the Smokers and Friends and LUBM learning and noise experiments. Real-valued logic typically bounds truth values to a defined range, which necessitates the use of clamping operations on the results. Automatic differentiation provides clamps that can perform the value clamping, but it also nullifies the associated gradient which can disable learning of involved parameters. Gradient-transparent clamping addresses this by fully recovering gradients outside of the allowable region by utilizing computations detached from the gradient-tracking computational graph to assert the clamping while keeping the original gradient information.

G.1 Clamping in neural networks

G.1.1 Smooth bounded activations

Neuron activation functions based on the logistic function, including sigmoid $(1 + e^{-x})^{-1}$ and tanh $(e^x - e^{-x})/(e^x + e^{-x})$ ensure outputs are in a bounded range, typically $[-1, 1]$ or $[0, 1]$, while ensuring differentiability across the entire domain. Bounded neuron activations ensure that values in the neural network don't grow too large and that there is a degree of interpretability at the output of neurons such as for binary classifiers or real-valued logic.

Smooth bounded activation functions $f()$ like sigmoid and tanh have two-sided saturation where gradients $\lim_{x \rightarrow \infty} \partial f(x)/\partial x = 0$ tend to zero in the extremes. The vanishing gradient problem affects learning with these bounded activation functions where neurons are saturated or where the chain rule in deep networks produce a negligible product of small gradients. The negative effect of vanishing gradients is that it can significantly attenuate signals that gradient descent depends upon for learning, effectively shutting down learning at some neurons. Residual connection is a solution in deep neural networks that skips over a stride of layers to shorten the number of interacting gradients and reduce the vanishing gradient problem. Another approach is to choose an activation function that does not have small gradients, like the ReLU.

G.1.2 ReLU

A rectified linear unit (ReLU) $\max(0, x)$ has a one-sided saturation of zero where $x < 0$ with an associated gradient of 0, and a linearity with gradient 1 otherwise. ReLU can address the vanishing and exploding gradient problems since its gradients can only be 0 or 1. However, this activation is affected by the “dying ReLU” problem where, if a preactivation distribution is limited to the negative domain, then the input gradients never propagate through the ReLU such that it cannot learn. Leaky ReLU and various forms of parameterized ReLU have been proposed to address the absence of negative domain gradients.

The lower bound on the ReLU output can be shown to be useful when neurons are viewed as concept detectors that give a degree of presence of a feature through the linearity, but only a uniform statement about the absence of a feature through the rectification. The ReLU lower bound can also possibly prevent correlated neurons by not propagating negative values corresponding to degrees of absence of a learnt feature. The ability to stop signal propagation could also reduce noise and extraneous information replication in the network.

The computational benefits of ReLU during learning is attributed to the gradient sparsity introduced by its zero gradients, which means that gradient calculation only needs to operate on a subset of the neurons. In addition, the two possible gradients of ReLU are constant scalars and no involved computation is required to determine its gradients.

G.1.3 Information and gradient dropout

Neural networks are universal function approximators that can embody functions of any complexity if provided with adequate capacity and a large enough set of neurons and interconnections between the neurons. Determination of the exact capacity required for arbitrary functions could be an intractable problem, so normally neural networks are given excess capacity or grow larger over a series of hyperparameter optimizations.

The problem of overfitting is where a trained neural network can not reach its objective performance on previously unseen data, so it can fail to generalize. Redundancy or excess capacity in neural networks allow for overfitting where the input training data could be memorized to optimize the stated objective. Various regularization methods like data augmentation and dropout have been proposed to prevent overfitting, which introduce random variations in the input data in the case of data augmentation or randomly remove neurons during training epochs.

Bounded activation functions that can saturate to an output and gradient of 0, such as sigmoid and ReLU could be considered to be performing a combination of information and gradient dropout. This can act as a regularization method that effectively removes capacity or disconnects certain neurons under the conditions of 0 saturation. The empirical success of dropout and ReLU has been attributed to the regularizing effects of the sparsity these methods introduce.

G.2 Gradient-transparent clamping

G.2.1 Clamping in automatic differentiation

The lower bound on ReLU $\max(0, x)$ performed through the clamp, min or max functions of automatic differentiation systems typically disconnects x from this node in the computational graph and replaces it with a new node valued 0 with no prior computational history attached. This means that the loss gradient could be $\partial f(x)/\partial x = 0$ even though x has been involved in the determination of the clamping outcome. This gradient statement says that any change to x will leave the output unchanged, which is no longer the case if an update to x is sufficiently large to bring it back to the allowable domain.

Severing the argument when clamping so that it is no longer represented in the computation could introduce inaccuracies in the interpretation of calculated gradients. If the subdifferential for 0 is set to the right-hand discontinuity, so that 0 gives the 1 gradient of the linearity, then it can state that the ReLU will decrease to a negative value for gradient descent at a linear rate. So if $x = 0$ the projected result of gradient descent with a step of $s = -0.1$ could be $x' = x + s = -0.1$ given a linear gradient, although this update could not materialize since it is already clamped at its starting value. So non-zero gradients provided do not give guidance on when the clamp could be encountered, which shows that even the normal application of clamping could result in incorrect gradient interpretations especially if function linearity is assumed.

Gradients provided by automatic differentiation both inside and outside of the clamped region are then subject to interpretation, and there is reliance on the learning update mechanisms to manage step sizes and momentum to navigate the pathological loss landscape introduced by clamping. Otherwise clamping can inform learning with gradients that indicate the loss can be changed by updating a parameter, even though clamping is then turned on at a fraction of the parameter step size. Clamping could also indicate that no learning is possible when the output is currently clamped, even though clamping turns off for a small parameter step in the right direction such that it does learn.

G.2.2 Information dropout and gradient transparency

Bounded activation functions that saturate at 0 can be said to perform regularization under certain pre-activation conditions by stopping the propagation of dynamic information and gradients. Approaches to information and gradient dropout are especially relevant to neural networks with redundant capacity

as a means of introducing sparsity to preventing overfitting and to address the vanishing gradient problem.

In the case of LNN as a one-to-one mapping of the syntax tree of a logical program, there is a significant difference to normal neural networks where hidden neurons do not necessarily relate to external concepts and where hidden layer widths can be arbitrarily sized. The problems of overfitting and dealing with redundancy are thus not of primary concern with LNN, although logical neurons and subnetworks could be copied to expand parameter capacity. The motivation for introducing gradient sparsity for normal neural networks through ReLU does not readily apply to LNN.

The vanishing gradient problem can be altogether avoided in LNN for certain loss terms that can be arrested neuron-wise so that a shorter gradient explanation can be calculated. However, it can be more complex for combined losses with supervised task-specific objectives calculated simultaneously over different batch dimensions representing alternate universes of truth value assignments. Smooth activations should then still be avoided even for LNN to address the vanishing gradient problem.

Information dropout through clamping in real-valued logic serves to keep truth values within the interpretable permissible range of the logic. Łukasiewicz logic applies clamping aggressively to ensure logical soundness, so the logic operation output values need to be bounded. However, the associated gradient outside of the clamped region need not necessarily be bounded as well. In fact, the severing of the gradient outside the clamp prevents learning for a significant part of the operating range of the logical neuron activation.

Gradient-transparent clamping provides information dropout or value clamping while also leaving gradients unaffected so that any gradient calculation effectively ignores clamping. The benefit is that the primary purpose of clamping is still fulfilled, namely clamping output values, but that the full output domain retains gradients as if no clamp was applied. The “dying ReLU” problem can also be solved with gradient-transparent clamping, since learning can receive gradients across the full operating range such that gradient descent always has the optimization direction available for a parameter.

The perceived errors this would introduce would be of exactly the same nature as those made by also clamping the output value. In particular, a gradient inside the bounds could indicate that the output will change even when the clamp then immediately applies into the update step, yet this was not reflected in the gradient. Similarly, a gradient outside the bounds given by gradient-transparent clamping could indicate an output change, but then the clamp still applies after the update step. The learning optimizer should manage the step sizes and momentum to ensure robust learning under these scenarios, both with and without gradient-transparency.

G.2.3 Gradient-transparent clamping

Automatic differentiation normally severs a computed node when clamping the output and substitutes it with a new node representing the applied bound. Removing the computed node also removes trace of its involvement in the current output, which is undesirable as it was indeed part of the calculation. The idea is then to retain the input to the clamping operation as part of the resultant node in the computation graph. This necessitates the definition of a new clamping function that performs value clamping while keeping gradients as if no clamp was applied.

For a lower bound clamp at $x_{\min}$ the value clamping is performed when $x < x_{\min}$ to give an output of $x_{\min}$. This can be calculated as $x - \min(0, x^* - x_{\min})$ where x^* denotes a copy of x that has been detached from the computational graph and thus carries no history. By obtaining the disconnected value x^* the value clamp can still be applied without the destructive interference otherwise caused if a tracked copy of the same node was used. Automatic differentiation tools normally do allow for detached value replication, and there are various capabilities that allow for value copy and reuse. The superscript (1) denotes that gradients in the clamped regions are unscaled, as opposed to the linear scaling performed by gradient supplantation that will be explained later.

$$\min(x_{\max}, x)^{(1)} = x - \max(0, x^* - x_{\max}), \quad (100)$$

$$\max(x_{\min}, x)^{(1)} = x - \min(0, x^* - x_{\min}). \quad (101)$$

Clamping can then be replaced with gradient-transparent clamping to recover gradients across the entire output range while still providing the intended value clamping. The provision here is that the